

MEPL Nepal

Bijulibazar, Kathmandu 10

Website: www.meplnepal.com

(977) 5245074

Sambridda Ranabhat

Automation Engineer (Intern)

Peristaltic Pump

May 14, 2026

Overview

This project involved the design and implementation of an automated peristaltic pump dispensing system intended for precise low-volume liquid delivery applications, particularly in medical and laboratory environments such as ophthalmic solution (eye-drop) measurement and dispensing. The system was developed using a stepper motor-driven peristaltic pump controlled through a PLC-HMI integrated automation platform.

The primary objective of the project was to achieve accurate and repeatable fluid dispensing by controlling the rotational movement of the peristaltic pump using a programmable logic controller (PLC), stepper motor driver, and human-machine interface (HMI). The system allows the operator to input a desired dispensing volume through the HMI, after which the PLC executes the required pulse control sequence to drive the stepper motor for a calibrated number of rotations corresponding to the target fluid volume.

The hardware architecture of the system consisted of a peristaltic pump coupled with a stepper motor and controlled through a DP508D stepper motor driver. A Coolmay PLC-HMI unit was utilized as the central control platform, enabling both machine control and user interaction within a compact integrated system. Additionally, a foot switch mechanism was incorporated to provide convenient hands-free operation during dispensing procedures.

The project also included the calibration of pump output with respect to motor rotation and pulse count to ensure dispensing accuracy and repeatability. Ladder logic programming was developed for motor control, pulse generation, user input handling, and operational sequencing. The HMI interface was configured to provide a simple and intuitive operating environment for entering dispensing parameters and initiating the pumping cycle.

Overall, the project demonstrates the practical application of industrial automation concepts including PLC programming, HMI development, stepper motor control, pulse-based motion control, and process calibration. The completed system provides a cost-effective and flexible solution for precision liquid dispensing applications and serves as a strong foundation for future development in automated dosing and medical fluid handling systems.

Problem Statement

Precise low-volume liquid dispensing is an important requirement in medical, laboratory, and industrial applications, particularly in processes involving ophthalmic solutions and other sensitive fluids. Traditional manual dispensing methods often suffer from inconsistencies caused by human error, uneven flow rates, and poor repeatability, making them unsuitable for applications requiring high accuracy and controlled dosing.

The challenge addressed in this project was the development of a compact and reliable automated dispensing system capable of delivering predefined fluid volumes with improved precision and repeatability. The system was required to provide user-friendly operation while maintaining accurate control over the dispensing process.

To address these issues, an automated peristaltic pump control system was designed using a PLC-HMI integrated controller, stepper motor drive, and stepper motor-based motion control. The system enables operators to input the required dispensing volume through the HMI interface, after which the PLC controls the stepper motor rotation through calibrated pulse control to achieve the desired liquid output.

Additionally, the system aimed to improve operational convenience and process consistency through the integration of a foot switch triggering mechanism and programmable automation

logic. The project focused on achieving accurate dispensing, repeatable performance, and simplified user interaction while utilizing cost-effective industrial automation hardware.

Objectives

- To automate the liquid dispensing process using a PLC-HMI integrated control system.
- To control the rotational movement of a peristaltic pump using a stepper motor and stepper motor driver.
- To enable user-defined volume input through the HMI interface for flexible dispensing operation.
- To establish a calibrated relationship between motor rotation and liquid output volume for accurate dispensing.
- To implement pulse-based motion control logic for precise stepper motor positioning and operation.
- To integrate a foot switch mechanism for convenient and hands-free system triggering.
- To improve dispensing consistency, repeatability, and operational efficiency compared to manual dispensing methods.
- To gain practical experience in industrial automation technologies including PLC programming, HMI development, motor control, and system integration.

System Overview

Instruments

S.N	Description	Qty
1	Peristaltic Pump	1
2	Coolmay PLC HMI	1
3	24V Power Supply	1
4	Stepper Motor Driver	1
5	Foot Controlled Switch	1

Logic

During the development of the dispensing system, an experimental calibration-based control methodology was adopted instead of relying solely on theoretical fluid flow calculations. Although the dispensing volume of a peristaltic pump can theoretically be estimated using parameters such as tube cross-sectional area, flow rate, tubing dimensions, and rotational displacement, such methods may introduce inaccuracies due to practical operating conditions.

In real-world operation, factors such as tubing elasticity, roller compression, fluid characteristics, and mechanical tolerances affect the actual output volume of the peristaltic pump. Therefore, a simplified and more practical calibration approach was implemented to improve dispensing reliability and ease of system development.

The control logic was based on the assumption of a constant proportional relationship between the stepper motor pulse count and the dispensed liquid volume. A calibration constant (K-factor) was experimentally determined by operating the pump for known motor pulse counts and measuring the resulting fluid output volume.

The relationship was represented as:

$$\text{Dispensed Volume} = K \times \text{Pulse Count}$$

$$\text{Pulse Count} = \text{Desired Volume} / K$$

Where:

- **K** represents the experimentally obtained volume-per-pulse calibration constant.

Using this method, the PLC program converts the user-entered target volume into the required pulse count for the stepper motor. The pulse output instruction then drives the motor for the calculated number of steps to achieve the desired liquid dispensing volume.

This calibration-based approach simplified the implementation process while providing sufficient accuracy, repeatability, and flexibility for the intended application. It also allowed easy

recalibration and adjustment without modifying the core control logic or performing complex fluid dynamic calculations.

Programming Methodology

The control logic for the automated peristaltic pump system was developed using ladder programming on the PLC integrated within the Coolmay HMI-PLC platform. The programming methodology focused on achieving accurate and repeatable liquid dispensing through pulse-based stepper motor control.

The overall operating sequence begins with the operator entering the required dispensing volume through the HMI interface. The entered value is stored in PLC memory registers and used as the reference value for the dispensing cycle. A foot switch was integrated as the triggering mechanism to initiate pump operation, allowing convenient hands-free control during operation.

The dispensing principle was based on the calibrated relationship between the stepper motor rotation and the liquid output volume of the peristaltic pump. Since the stepper motor rotates in discrete steps, the PLC was programmed to generate a specific number of high-speed output pulses corresponding to the desired dispensing volume.

The programming logic involved the following major steps:

1. Reading the desired volume input from the HMI.
2. Converting the entered volume into the required motor pulse count using experimentally determined calibration values.
3. Monitoring the foot switch input condition to trigger the dispensing cycle.
4. Activating the stepper motor driver by generating high-speed pulse outputs from the PLC.
5. Running the motor for the calculated pulse count to rotate the peristaltic pump by the required amount.
6. Stopping the motor automatically after completion of the dispensing cycle.

The PLC outputs were connected to the pulse (PUL) and direction (DIR) input terminals of the DP508D stepper motor driver. The pulse output controlled the number of motor steps and

rotational speed, while the direction output defined the motor rotation direction. Fixed direction control was used since the application only required forward dispensing operation.

To achieve dispensing accuracy, calibration testing was performed by operating the pump for known pulse counts and measuring the corresponding liquid output volume. The obtained calibration factor was then incorporated into the ladder logic to establish accurate volume-to-pulse conversion.

Additional programming considerations included operational interlocking, prevention of repeated triggering during active cycles, and stable pulse generation for reliable motor operation. The HMI interface was designed to provide a simple operating environment with controls for volume entry and system operation.

The completed logic successfully integrated PLC control, HMI interaction, stepper motor pulse control, and calibrated dispensing operation into a compact automated fluid dispensing system.

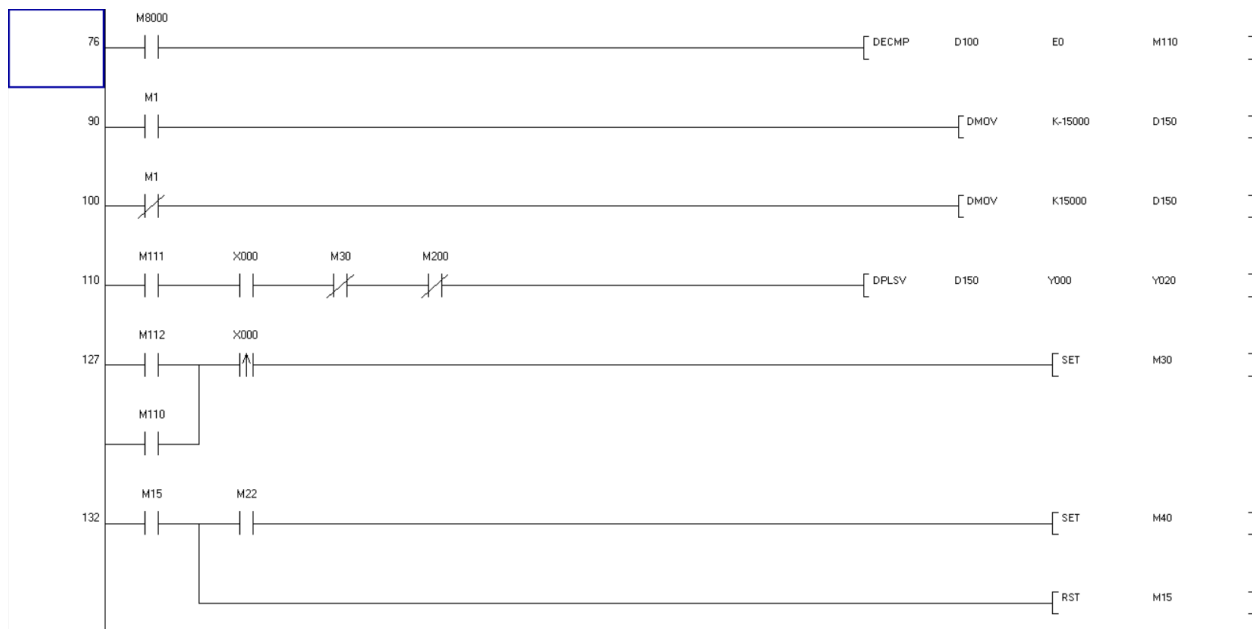
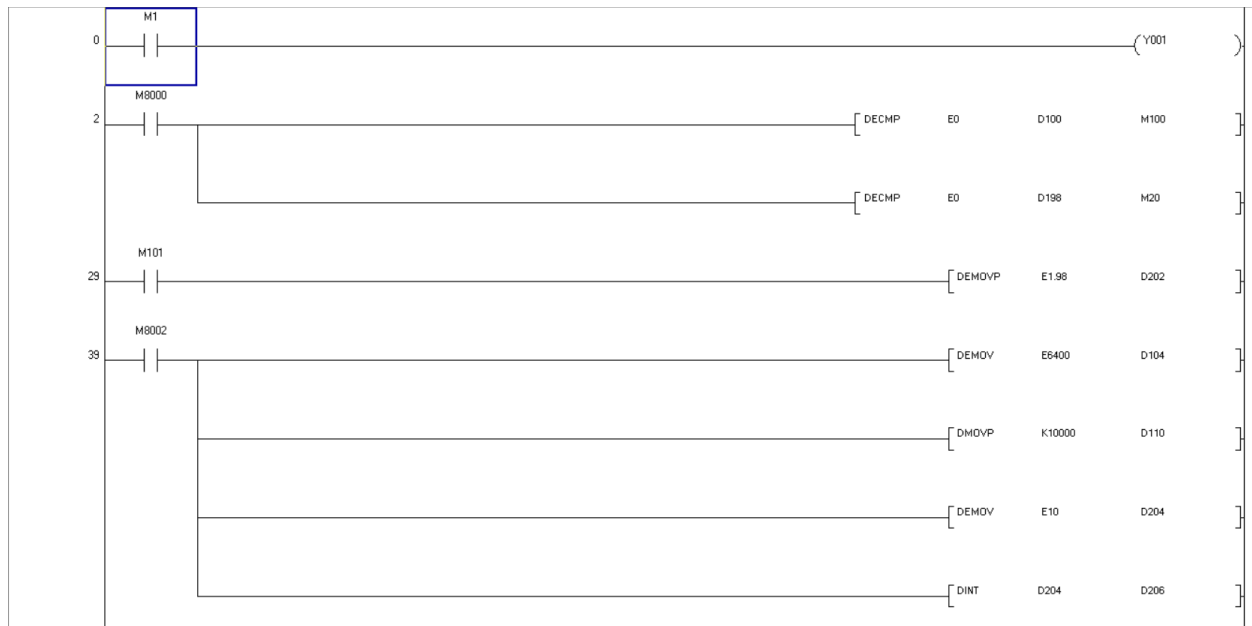
Software & Hardware Infrastructure

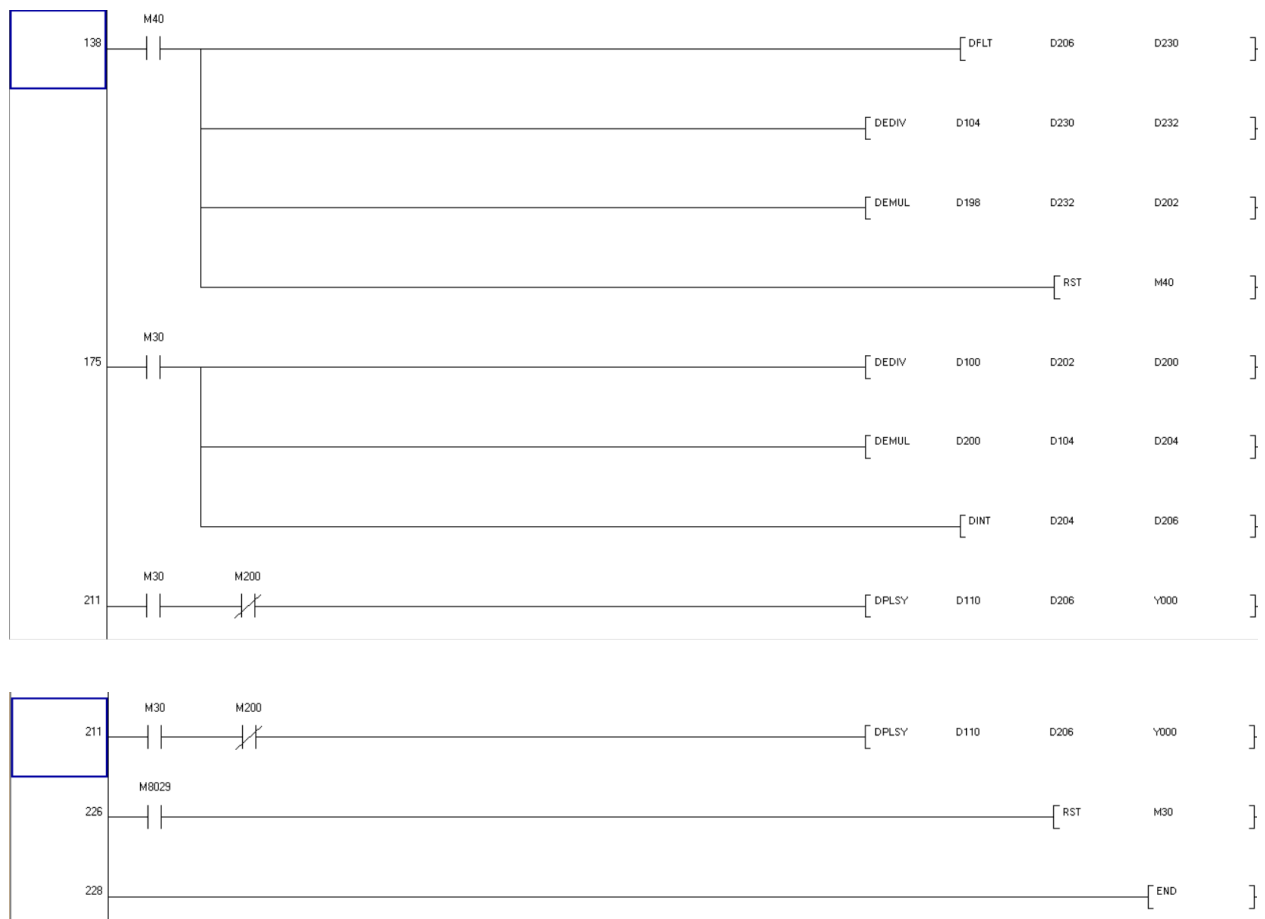
Software

To realize the automated and manual control loops of the dosing pump system, two core industrial software suites were utilized:

- **GX Works2 (Mitsubishi Electric):** Used as the central PLC programming environment. The system logic was developed using standard Ladder Logic (LD) optimized for an FX3U-compatible architecture. High-speed pulse output instructions, standard 16-bit/32-bit integer math, and floating-point data manipulations were executed here to control motor velocity and accurate volume distribution.
- **mView (HMI Design Software):** Used to design the operator interface (Human-Machine Interface). mView handled the graphical representation of machine states, directional controls, live data monitoring, and numeric entry objects for operator-driven calibration. Communication variables (M-bits and D-registers) were mapped directly to establish a real-time data handshake between the HMI screen and the PLC registers.

Ladder Logic Architecture





Key logic breakdown:

Master Direction Control (Rung 0): A global directional steering philosophy was implemented using bit **M1**. Rather than allowing manual and automatic routines to independently manipulate physical outputs, **M1** acts as the exclusive controller for the direction output (**Y001**). This prevents logic conflicts and ensures consistent clockwise/anticlockwise tracking across all modes.

System Initialization (Rung 20): Utilizing the first-scan pulse **M8002**, critical system parameters—including default pulse velocity (**D110**), floating-point calibration constants, and initial dosing values—are safely written into the data registers upon CPU boot to ensure a predictable machine state.

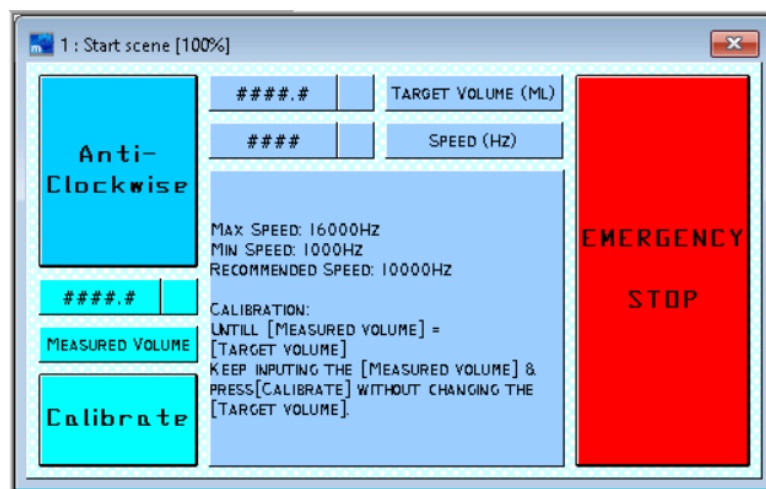
Manual & Priming Mode (Rungs 71–91): Manual override utilizes the **DPLSV** (Deleterious/Variable Speed Pulse Output) instruction triggered by **M111**. To eliminate double-coil conflict with the master direction switch, the internal instruction sign bit was rerouted to an unused virtual output (**Y020**), leaving physical control exclusively to the **M1** rung.

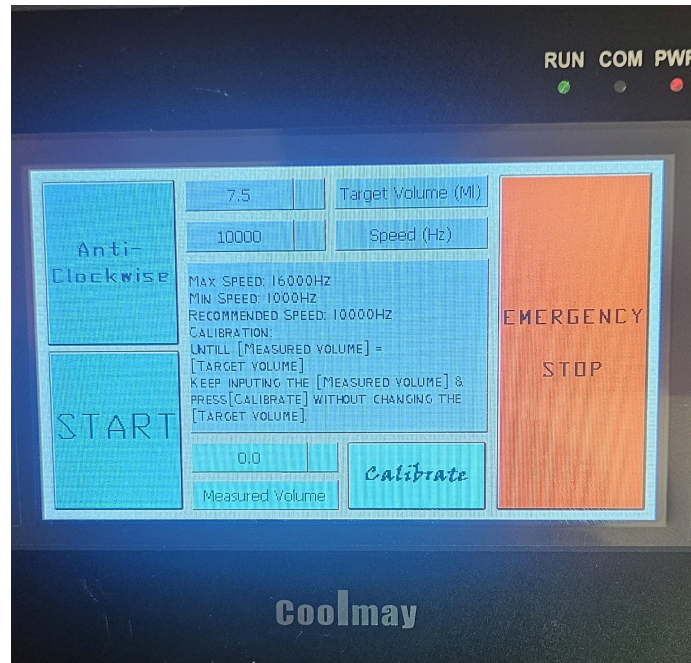
Automated Dosing Loop (Rungs 174–210): The automated batching routine reads the user's target volume from **D100**, processes it against the saved calibration K-factor (**D202**), converts the floating-point volume into an exact target pulse integer (**D206**), and triggers the **DPLSY** instruction. The loop self-terminates precisely on the target pulse count via the execution complete flag (**M8029**).

Safe Calibration Routine (Rungs 120–125): To protect the controller from math execution faults (such as a processor-halting divide-by-zero error), a conditional gate was constructed. Using binary floating-point comparisons (**DECMP**), the calibration sequence (**M40**) is completely blocked if the operator-entered measured volume in register **D120** is equal to **0.0**. It only commits the new K-factor calculations when a valid, positive non-zero float is present.

HMI Design

The user interface was engineered in mView with an emphasis on minimalist operation, clear state feedback, and foolproof industrial safety.





Interface highlight

Register Mapping Synchronicity: All numeric input fields and state buttons are bound tightly to the PLC memory map (e.g., target volume bound to **D100**, calibration data entry bound to **D120**). Data entries are defined as 32-bit floating-point structures to ensure accuracy matches the math depth inside the PLC backend.

Input Protection Gating: HMI action types were carefully selected based on function. Command triggers (like the "Calibrate" button) utilize momentary pulse actions to prevent "stuck bits" or infinite loop execution inside the PLC logic scan.

Retentive vs. Volatile Memory Separation: To protect operator settings, the system separates field entries. Volatile registers are assigned to temporary variables, while the calculated system calibration constant is stored permanently in the PLC's retentive memory range (**D202**), ensuring the machine retains its precision even after a complete power loss.

System modes of operation and user experience

This dosing system was engineered with a clear operational split between manual utility and automated precision. Below is the technical breakdown of how the system functions under

different production requirements and the underlying mathematical mechanics of the calibration engine.

Continuous Dispensing Mode (Non-Stop/Manual Operation)

Continuous mode is designed for system priming, line purging, or indefinite liquid transfer where a predetermined volume is not required.

- **Operational Workflow:** 1. The operator interacts with the global direction toggle switch on the mView HMI (M1). Flipping the switch determines the rotation: OFF sets the pump to Clockwise (Forward), and ON sets it to Anticlockwise (Reverse). This routes the physical state of the directional output wire (Y001).
2. The operator presses and holds the manual run button (M111).
- **Behind the Logic:** The PLC catches the rising edge of the button press and loads a constant pulse velocity of 15,000pulses/sec into data register D150. The DPLSV (Variable Speed Pulse Output) instruction immediately commands high-speed output pulse stream Y000 to drive the motor.
- **User Experience (UX) Feel:** The response is instantaneous. Because the direction flag is decoupled from the pulse command loop and managed globally, the pump handles seamlessly without lagging or direction-switching logic conflicts. Releasing the button instantly drops the execution rung, terminating the pulse train with zero trailing or slipping.

Batch Dispensing Mode (Target Volume Operation)

Batch mode provides automated, volumetric dosing based on precise physical pulse tracking.

- **Operational Workflow:**
 1. The operator enters the desired target volume (e.g., 10.0 mL) into the numeric entry field bound to D100.
 2. The operator verifies the master direction switch (M1) is configured for the production line requirements.
 3. The operator presses the momentary Auto Start HMI button (M112).

- Behind the Logic: The PLC takes the user-defined volume from D100 and calculates the exact number of pulses required using the relationship:

$$\text{Target Pulses (D206)} = \text{Target Volume (D100)} / \text{System K-Factor (D202)}$$

- The resulting integer is fed as a hard limit into the DPLSY (Pulse Output with Pulse Count) instruction. The pump ramps immediately to its automated velocity profile of 10,000 pulses/sec (D110).
- User Experience (UX) Feel: The system behaves as a "set-and-forget" routine. The operator receives live feedback while the pump runs. The moment the internal high-speed hardware counter matches the calculated value in D206, the PLC's hardware execution flag (M8029) trips, self-terminating the cycle and resetting the process interlocks.

The Calibration Mode

Over time or when changing liquids of different viscosities, the physical output of a pump drifts from its theoretical profile. This system features an integrated, closed-loop calibration routine to recalculate the scaling constant (K-Factor) without requiring manual programming.

Step-by-Step Operator Procedure:

1. The operator runs a standard test batch expecting a specific volume (e.g. 10.0 mL).
2. The operator physically measures the fluid output using a graduated cylinder (e.g., actual yield is 9.5 mL).
3. The operator inputs the value 9.5 into the Measured Volume numeric entry box on the HMI (D120).
4. The operator hits the momentary Calibrate button (M15). The system updates, locks the new configuration, and the entry field is wiped clean back to 0.0.

The calibration engine uses 32-bit floating-point math blocks to avoid rounding errors that degrade performance over thousands of production cycles. When M15 is triggered, the system checks the input register using floating-point comparison (DECMP).

If the value is verified as valid and non-zero ($D120 > 0.0$), the calibration flag (M40) executes the following mathematical sequence in a single program scan:

1. Pulse-to-Float Conversion (DFLT): The raw integer pulses tracked during the test run (D206) are converted into a 32-bit float and stored in D230 to allow precise division.
2. Ratio Extraction (DEDIV): The system calculates the discrepancy ratio by dividing the target pulse density by the recorded pulses:

Pulse Ratio (D232) = Theoretical Target Base Volume/Actual Extrapolated Floating pulse

3. K-Factor Recalculation (DEMUL): The new scaling factor is generated by multiplying the operator's measured fluid volume against the pulse ratio:

New KFactor (D202) = Measured Volume (D120) x Pulse Ratio (D232)

This new value over-writes the old scaling constant in **D202**. Because **D202** sits in the retentive memory block of the FX3U architecture, this precision value is permanently saved, protecting the machine profile against unexpected factory power disruptions.

Engineering Challenges & Troubleshooting solutions

Developing the dosing pump system exposed several critical integration anomalies between the PLC execution cycle, HMI communication asynchronous behavior, and floating-point math constraints. Below are the key engineering hurdles encountered and the collaborative strategies implemented to resolve them.

Challenge 1: Asynchronous Communication "Race Conditions" (The Stuck HMI Button)

- **The Problem:** During testing of the calibration trigger sequence, the HMI button bound to bit **M15** would visually "hang" or freeze in the active state, causing the ladder logic monitoring system to reflect a permanent logic-high conflict. The PLC's **RST M15** cleanup command was failing to reset the contact.
- **The Root Cause Analysis:** The HMI button action was configured as a persistent "**Set ON**" signal. Because HMI cyclic communication runs asynchronously to the PLC task execution loop, a classic "race condition" occurred: The operator pressed the button,

writing a **1** to memory. The PLC scanned the rung, processed the math, and executed a **RST M15** to switch it to **0**. However, on the next HMI refresh frame, the HMI panel read the **0**, assumed a status mismatch based on its "Set ON" state, and forcibly over-wrote it back to a **1**.

- **The Solution:** We diagnosed the handshake loop by mapping out the cycle timings. The HMI button's physical action driver was refactored in **mView** from an aggressive "Set ON" to a **Momentary** pulse action type. This stripped the HMI of its forcing control, allowing the operator's physical touch to transition the bit to a **1**, while giving the PLC's local **RST** instruction absolute authority to instantly clean up and drop the bit at the end of the scan.

Challenge 2: Floating-Point Corruption via Misaligned Data Type Frameworks

- **The Problem:** When loading calibration parameters and displaying measured volumetric calculations, the HMI numeric fields frequently displayed scrambled data arrays, unexpected overflow characters, or massive random numbers, despite the PLC registers housing valid values.
- **The Root Cause Analysis:** The data pipelines were cross-threaded. Inside GX Works2, the math blocks were handling precision division and multiplication using 32-bit floating-point commands (DEDIV, DEMUL, DECOMP). However, an audit of the mView configuration windows revealed that the Numeric Entry data mapping format was set to [32Bit] Unsigned Integer while its display layout was set to 32 Bits Float. Point. The HMI was interpreting raw, IEEE-754 encoded floating-point binary bits as standard integer packets, corrupting the display presentation.
- **The Solution:** The database architecture was standardized. The HMI register mapping profile was reconfigured to perfectly align with the controller's backend math depth. By changing the HMI Data Format properties to explicitly handle [32Bit] Float, the telemetry alignment was restored, enabling flawless decimal precision readings down to 0.1 mL.

Challenge 3: System Fault Mitigation (Preventing CPU Math Halts)

- **The Problem:** If an operator accidentally clicked the calibration button while the physical data tracking field was empty 0.0 mL, the calculation loop would process the blank field,

leading to severe computational instabilities or risking a catastrophic CPU math halt error.

- The Root Cause Analysis: The original calibration instruction sequence directly divided the target parameters against the input register. In standard programming, passing an absolute zero through a divider creates an infinite loop or an illegal operation fault, causing industrial PLC processors to fault out and trip the emergency safety stop.
- The Solution: We engineered a defensive software gate using binary floating-point comparisons. A DECOMP block was introduced at the absolute front-end of the calibration ladder rung. The logic evaluates the register data type conditions; if the measured volume is equal to E0 (floating-point zero), the M21 status bit trips, bypassing the entire execution branch and blocking the math sequence from firing. The calculation loop is strictly interlocked, only opening the execution path (M22) when a valid, positive, non-zero number is detected.

Challenge 4: Data "Sticky" Drift on Power Cycles

- The Problem: Early iterations utilized retentive registers (D250 range) for temporary calibration data entry storage. If an operator aborted a calibration mid-sequence or turned off the control panel cabinet, the partial numbers remained permanently stamped in the register memory, causing confusing residue entries upon the next machine boot.
- The Root Cause Analysis: Registers located in the D200–D511 allocation maps within the FX3U architecture are hardware-latched via an internal battery backup. While this is necessary for system constants (like the final K-Factor), applying it to transient data entries violates industrial safety standards.
- The Solution: The variable memory map was restructured. The temporary user entry field was completely migrated to D120, a strictly volatile, non-retentive register block. To guarantee a zero-baseline reset, an initialization line driven by the CPU's First Scan Pulse (M8002) was programmed to overwrite D120 with a clean E0 at startup, ensuring a fresh, uncorrupted system state every single time the system cycles power.

Testing & Validation

Functional Logic and Communication Handshake Validation

Before introducing liquid into the physical assembly, the control system underwent rigorous dry-run testing using real-time bit monitoring inside GX Works2. The first critical validation focused on eliminating any asynchronous communication bottlenecks between the mView HMI and the PLC execution cycle. By tracking the state of the calibration command bit across consecutive program scans, the handshake mechanism was observed following its configuration change to a momentary pulse action. The local PLC reset instruction demonstrated absolute authority, instantly clearing the tracking bit within a single scan loop and allowing the HMI graphical interface to dynamically pop back up with zero visual lag or register lockups. Additionally, the global direction framework was heavily stressed by rapidly toggling the directional routing switch during transitions between manual and automatic operational modes. The physical direction output consistently maintained exclusive tracking of the master flag, proving that decoupling the pulse stream from direction control successfully eliminated standard double-coil logic conflicts.

Software Interlock and Fault Mitigation Testing

System safety and runtime stability were verified by challenging the computational boundaries of the 32-bit floating-point math blocks. The primary objective was ensuring absolute protection against an operational processor halt, which typically occurs in industrial controllers during an illegal divide-by-zero sequence. Testing was conducted by intentionally leaving the temporary calibration register blank at a baseline value of exactly zero and repeatedly executing the calibration command trigger. The binary floating-point comparison instruction successfully trapped the invalid zero input on every attempt, completely blocking the execution path to the division and multiplication blocks and routing the logic through a safe bypass bit instead. This defensive programming sequence effectively proved that the system is entirely foolproof against accidental operator input errors, ensuring uninterrupted processor runtime under field conditions.

Volumetric Calibration and Multi-Point Linearity Testing

To validate the accuracy and linearity of the dosing pump control system, empirical wet-testing was conducted using a high-precision, graduated 20 mL laboratory measuring vessel. The

verification process was split into two primary phases: baseline calibration alignment and multi-point volumetric testing across the pump's operating spectrum. The initial baseline test was executed by commanding an automated batch of 10.0 mL via the mView HMI using the factory-default pulse-scaling factor. Upon measuring the physical output, any initial mechanical displacement deviations caused by tubing tolerances, fluid viscosity, or pump slip were recorded, and the empirical value was submitted into the calibration data entry register. This triggered the internal 32-bit floating-point math blocks to automatically generate a corrected, retentive K-factor, successfully aligning the PLC's pulse-train generation with physical volumetric distribution.

Following the baseline calibration at the 10.0 mL mark, a multi-point verification matrix was carried out to test the system's volumetric linearity and repeatability across a wide operating range. The system was commanded to dispense varying test batches at distributed intervals of 5.0 mL, 7.5 mL, 15.0 mL, and 20.0 mL. Testing across these fractional and full-scale boundaries is critical in fluid dynamics to ensure that the calculated scaling factor holds true regardless of whether the pump is operating at the lower or upper limits of its intended cycle capacity. The results of the multi-point validation confirmed exceptional system precision and structural stability. Across all tested volumes, the physical liquid lines aligned flawlessly with the graduations on the measuring vessel, demonstrating a tracking precision within an estimated tolerance of ± 0.1 mL. This high level of alignment across asymmetric target volumes proves that the floating-point calibration engine successfully eliminated linear scaling errors, ensuring the dosing pump maintains highly repeatable, industrial-grade accuracy throughout its entire volume range.

Volatile Memory Alignment and Cold-Boot Verification

The final testing phase verified the environmental resilience of the controller's memory map during unexpected facility power disruptions. A strict validation routine was established by intentionally leaving transient data resting in the calibration input register and executing a forced hard-kill on the main control cabinet's breaker system. Upon restoring power, the initialization logic was monitored during the controller's cold-boot sequence. The CPU's first scan pulse successfully captured the initial program loop, forcing a floating-point zero instruction to completely overwrite the volatile register. This successfully reset the HMI numeric field to a baseline state while leaving the custom-calculated calibration constants perfectly intact within

their designated retentive memory addresses. This test confirmed total memory isolation, guaranteeing that the machine will safely power cycle without losing its precision profile or booting up with corrupt, stale data.